Cheating Videos Recognition

VGG16 CNN with Transfer Learning Report

Abanoub Shenouda Samy

Model	VGG16 CNN with Transfer Learning
Task	Binary frame classification: cheating vs normal
Test Accuracy	78.52% from the saved confusion matrix
Main Output	Reusable .keras model and live camera prediction notebooks

1. Project Overview

This project detects cheating behavior in exam hall videos or live camera streams. The system converts videos into frames, trains a Deep Learning model, and classifies each frame as cheating or normal.

- `cheating`: suspicious or cheating behavior is detected.
- `normal`: no cheating behavior is detected.

The live camera test runs for about 35 seconds and decides the final result based on the percentage of frames classified as cheating.

2. Data Preprocessing

Preprocessing is implemented in `Master_Preprocessing.ipynb`.

1. Read raw videos from `dataset/normal` and `dataset/cheating`.
2. Split the data at video level into train, validation, and test sets to reduce data leakage.
3. Extract frames from each video using `SAMPLE_FPS = 6`.
4. Resize all frames to `224x224` using padding to preserve the original aspect ratio.
5. Save individual frames and 16-frame sequences with stride 8.

Processed Data Distribution

Dataset	Class	Frames	Sequences
Train	cheating	4230	419
Train	normal	935	102
Validation	cheating	841	81
Validation	normal	245	26
Test	cheating	934	94
Test	normal	239	26

Important note: the dataset is imbalanced. The number of cheating samples is much higher than the number of normal samples.

3. Model Used

The main model is a CNN based on Transfer Learning using VGG16.

Item	Details
Training notebook	Abanoub_CNN_VGG16.ipynb
Saved model	abanoub_cheating_detection_results/best_model.keras
Base model	VGG16 pretrained on ImageNet, frozen, include_top=False

Item	Details
Final layer	Dense(1, sigmoid)

Model Architecture

- Input: 224x224x3 image
- VGG16 pretrained on ImageNet, include_top=False, frozen
- GlobalAveragePooling2D
- Dense(256) + L2 regularization + BatchNormalization + ReLU + Dropout(0.5)
- Dense(128) + L2 regularization + BatchNormalization + ReLU + Dropout(0.3)
- Dense(1, sigmoid)
- Output: probability of class 1

Training Configuration

Setting	Value
Image size	224x224
Batch size	32
Maximum epochs	100
Loss function	binary_crossentropy
Optimizer	Adam, learning rate 1e-3
Metric	accuracy
Regularization	Dropout, L2 regularization, BatchNormalization
Callbacks	ModelCheckpoint, EarlyStopping, ReduceLROnPlateau, CSVLogger
Class imbalance handling	class_weight

4. Input & Output

Training Input

The model receives RGB image frames with shape `224 x 224 x 3`.

Input folders: `processed_data/frames/train`, `processed_data/frames/val`, and `processed_data/frames/test`.

Training/Evaluation Output

The model outputs one sigmoid value between 0 and 1. The prediction threshold is 0.5.

The class order from `image_dataset_from_directory` is `['cheating', 'normal']`, so the output must be interpreted carefully if class names are changed.

Live Camera Output

Each captured frame returns a label and confidence. The final result is `CHEATING` if the cheating-frame ratio is greater than `CHEAT_THRESHOLD = 0.5`; otherwise it is `NORMAL`.

5. Results and Accuracy

The saved confusion matrix gives the following values:

True \ Predicted	cheating	normal
cheating	814	120
normal	132	107

Total test frames: 934 cheating + 239 normal = 1173 frames.

Correct predictions: 814 + 107 = 921.

Test accuracy: 921 / 1173 = 78.52%.

Additional Metrics

Class	Precision	Recall	F1-score
cheating	86.05%	87.15%	86.59%
normal	47.14%	44.77%	45.92%
Weighted average	78.12%	78.52%	78.31%

The model performs reasonably well at detecting cheating, but it is weaker on the normal class. This is mainly caused by class imbalance and the smaller number of normal examples.

6. Visuals

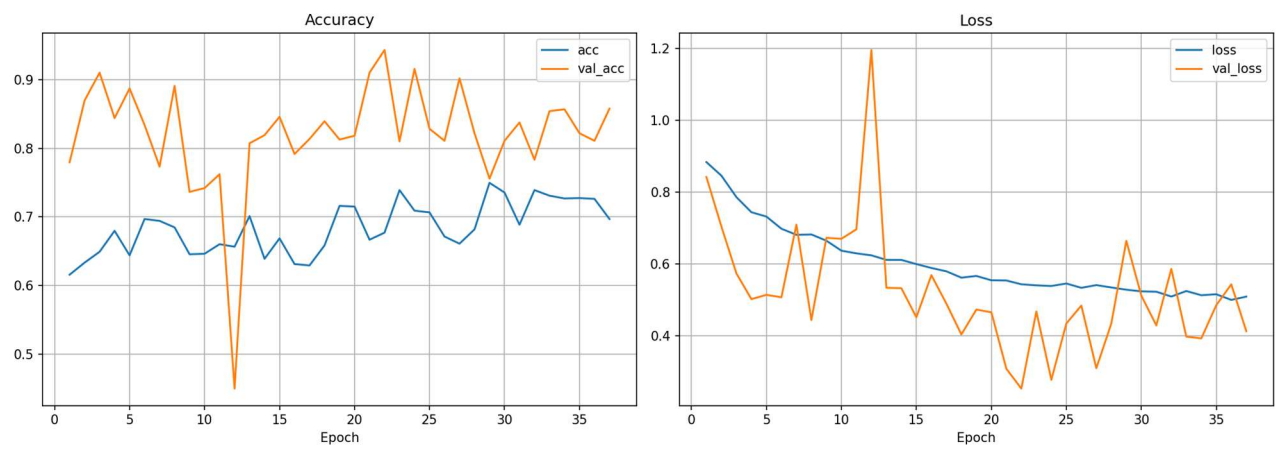

Figure 1. Training and validation accuracy/loss curves.

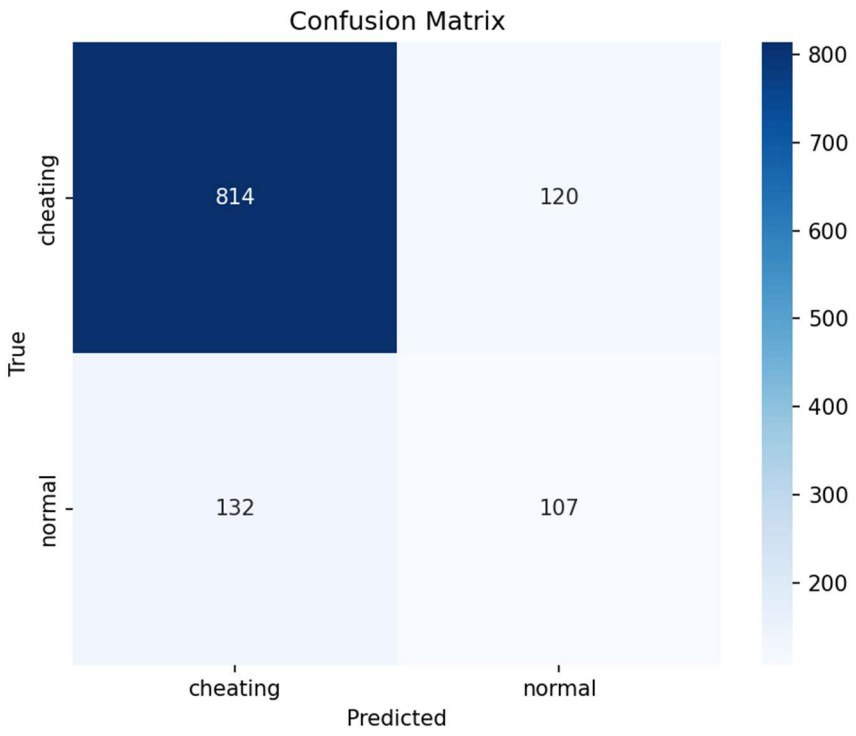

Figure 2. Confusion matrix on the test set.

7. Model Advantages

- Uses Transfer Learning from VGG16 and benefits from ImageNet pretrained visual features.
- Good baseline for image classification without training a CNN from scratch.
- Simple input format: one 224x224 frame.
- Data augmentation helps reduce overfitting.
- Dropout, L2 regularization, and BatchNormalization improve training stability.
- The trained model is saved as `.keras` and can be reused in the camera test.

8. Model Limitations

- VGG16 is relatively heavy compared with newer models such as MobileNet or EfficientNet.
- The model is frame-based and does not directly learn motion over time.
- Class imbalance clearly affects normal-class performance.
- Using a single frame can cause false positives when normal posture looks similar to cheating behavior.
- Normal-class recall is weak at about 44.77%.
- Validation curves are unstable, suggesting that the validation split is small or not balanced enough.

9. Problems Faced and Solutions

Problem	Description	Solution
Different video and frame sizes	Videos have different resolutions and aspect ratios.	Used `resize_with_padding` to convert frames to 224x224 while preserving aspect ratio.
Class imbalance	Cheating samples are much more frequent than normal samples.	Used class weights during training and data augmentation to increase variation.
Data leakage risk	Random frame splitting could place frames from the same video in both train and test.	The split was done at video level before saving processed data.
Unstable validation results	Validation accuracy and loss are unstable.	Used EarlyStopping and ReduceLROnPlateau, with a recommendation to increase validation data.
Camera testing differences	Camera access differs between Windows, Colab, and mobile streams.	Prepared OpenCV local testing, Colab JavaScript capture, and IP camera support.
Cheating type interpretation	The trained model is binary and does not truly identify the cheating type.	Used simple heuristics in the display notebook; a multi-class model is recommended for accurate type detection.

10. Conclusion

The project builds a complete pipeline: Raw Videos -> Preprocessing -> Frames/Sequences -> VGG16 CNN -> Evaluation -> Live Camera Test.

The current frame-level test accuracy is about 78.52%. The model is relatively strong at detecting cheating, but weaker at recognizing normal behavior due to class imbalance. The main improvement priorities are adding more normal data, trying lighter CNN architectures, and using a sequence-based model that can learn motion over time.